

A Reference Architecture for Social Media Intelligence Applications in the Cloud

Ivor D. Addo, Duc Do, Rong Ge, Sheikh I. Ahamed

Marquette University

Milwaukee WI, USA

{ivor.addo, duc.do, rong.ge, sheikh.ahamed}@marquette.edu

Abstract – As the social media upsurge of today continues to mount, opportunities to derive collective intelligence from online social networking (OSN) content sources are inevitably expected to grow. While enterprise organizations and research institutions make a dash for identifying rich insights and opportunities to tap into the millions of conversations and user profile relationships exposed by this new social-influenced big data phenomenon, architectural concerns regarding the storage and processing of large datasets unearthed by OSNs, along with performance, scalability, fault-tolerance, security, privacy, and high-availability solutions have become an area of concern for social media intelligence (SMI) solutions. In this literature, we present a reference architecture, for designing SMI solutions. In addition, we showcase two key case studies for SMI applications built on this architecture. Our selected case studies are focused on the analysis of User-Generated Content (i.e. with Sentiment Analysis in Twitter data) and Social Graph Influence (i.e. in a Facebook-influenced Movie Recommendations solution). We evaluate the ‘goodness-of-fit’ in applying our model to these case study solutions and present results from our performance evaluation of these cloud-hosted solutions across multiple cloud providers like Amazon AWS, Microsoft Azure and Google Cloud.

Index Terms – Reference Architecture, Social Media Intelligence, Social Insights, Big Data Analysis, Social Data Science, Cloud Architecture, Collective Intelligence.

I. INTRODUCTION

Indisputably, Social Media has created a platform for generating a wealth of data. With the advent of this ever so trendy communications channel, opportunities for extracting and utilizing insights from online social network (OSN) interactions is picking up very quickly. Enterprise organizations are racing to draw a competitive advantage from the collective intelligence that can be gained by examining social media discernments. In the same vein, researchers are attracted to the opportunities and inherent challenges that come along with the massive data collection, storage, and processing requirements that this phenomenon presents. At the same time, breakthroughs in cloud computing, “big data” solutions and novel machine learning algorithms have made significant contributions to opening up a new world order for deriving rich analytics from social media content and user profile interactions.

We refer to the unification of insights derived from social media sources as *social media intelligence* (SMI). Some of the prominent social media sources of today include Facebook, Twitter, YouTube, LinkedIn, Pinterest and

more. Recent SMI research studies and implementations range from statistical analysis aimed at drawing conclusions about user influence [1] and opinion mining [2] [3] on Twitter, to the optimization of targeted advertising and relevant search results ranking [4] based on Facebook insights, as well as, in some cases, enterprise solutions that leverage insights gleaned from social intranet applications like Jive, Yammer and SharePoint. In most cases the entities of interest in SMI analysis can be generalized as a variant of *social user profiles* and/or *social media content* as described in the Venn diagram in Figure 1.

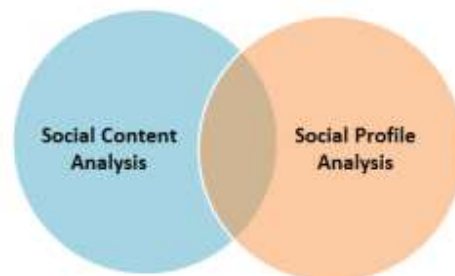


Figure 1. SMI Analysis Classifications

With the two SMI classifications presented above in mind, we embarked on the implementation of two case studies that cover the spectrum of SMI analysis. From a *Social Profile Analysis* standpoint, we examined what goes into building a movie recommendations cloud service that pulls social insights from the movie preferences of a given user’s Facebook friends as intelligence for ranking personalized movie suggestions. This solution was deployed as a web application service hosted on the Microsoft Azure cloud platform. In addition to this case study, we implemented a sentiment analysis solution for evaluating the impact of *Social Media Content*, that is, Twitter messages (or tweets) on a given brand’s reputation by utilizing the Amazon AWS cloud platform for *Elastic MapReduce* (EMR) including *Hadoop*.

With the proliferation of web application programming interface (API) solutions in our day, it is not surprising that various social media content sources expose web services for retrieving and analyzing social media data. Sustaining solutions that are built for data intensive SMI support comes with it, some architecturally significant concerns including performance, scalability, fault-tolerance, high-availability, disaster recovery, security, privacy and the

likes. In addition, fast and suitable machine learning algorithms are often at the heart of building high-performance SMI solutions which generalize based on the inherent patterns [5] in the nonrandom data presented in the social media contexts.

We recognize that as the evolution of social media continues to thrive, so will intelligence derived from social media sites (that is, SMI). The Adobe Digital Report [32] contends that brands are continually getting better at engaging their consumers through social media. Revenue opportunities in social media intelligence abound. It is not surprising that SMI solutions are increasingly employed in various industries. Astonishingly, there is currently no widely adopted reference architecture for governing the implementation of SMI applications. With our cloud-enabled reference architecture for SMI, we seek to provide researchers, software engineers, and IT architects with a playbook for jumpstarting their efforts at tapping into social intelligence to unearth the inherent myriad of opportunities.

Our Contributions

Drawing from our examination of recently published implementations of SMI solutions, coupled with our implementation of the case studies described earlier, we derived an optimal architectural design for building and sustaining large-scale SMI solutions that can exhibit high-scalability, high-availability and performance among other quality attributes. Our contributions take the form of the logical *Reference Architecture* presented in this literature. We consider this design artifact as a key contribution to research and enterprise communities since we are not aware of any existing study that provides a comprehensive and technology-agnostic design blueprint for SMI-related solutions. We share some lessons learned from our study to help other researchers and solution architects extend the template solution to suit their needs.

In the ensuing literature, we present our contributions by describing our motivation for the study along with some applicable SMI scenarios and their inherent architectural challenges in section II. The reference architecture is described in section III. Sections IV and V showcase the two case studies that we used to validate the “goodness-of-fit” for our proposed SMI reference architecture. Section VI describes our evaluation approach and results. Some of the related studies that inspired our work are presented in section VII. We conclude by summarizing our findings in section VIII.

II. MOTIVATION AND CHALLENGES

In the face of recent advancements in big data and cloud computing technologies, our research goals are well aligned to building high performance data intensive solutions that leverage collective intelligence [5][6] from various sources. Invariably, large-scale SMI solutions happen to offer a good platform for evaluating some of the architectural boundaries that we wish to investigate. In future studies, we

hope to extend some of the MapReduce and Hadoop-based scenarios that were used as case studies in this research work. Like most reference architectures [11], we are hopeful that our proposed strategy will promote re-use, minimize maintenance costs, and serves as a benchmark for SMI solution governance.

From the implementation of SMI for social influenced search results ranking on Microsoft’s Bing.com search engine [4], to widespread applications in delivering contextual and relevant advertisements on web sites, there is a wide spectrum of opportunities for harnessing large-scale SMI solutions to drive a competitive advantage. While there are several scenarios that can leverage social media influenced collective intelligence solutions, we are particularly drawn to evaluating data intensive and computing intensive case studies that cut across the two major classes of SMI entities (described in section 1). Some of the architectural concerns and requirements pertaining to these scenarios are described in the ensuing literature.

A. Social Influence in Movie Suggestions

A natural way to determine suggestions for movies that a given user might be interested in, could entail asking friends who have similar tastes as the user in question or people who knowingly or unknowingly appear to influence the user to provide recommendations. If that scenario is transposed into a world where most of our friends and other people who potentially share similar “tastes” in movies as we do, then perhaps harnessing the power of social intelligence to simulate an *always-on* web of recommendations based on preferences gleaned from similar users and our self-reported “friends” could be a very viable option. That, in essence, is the premise of this particular case study.

Architectural Concerns and Non-Functional Requirements

This interesting and useful SMI solution is not without significant architectural concerns. Some of the characteristic architectural concerns and non-functional requirements that will need to be addressed in a reference architecture that supports this type of solution include but are not limited to:

- **High Performance:** A large-scale implementation of this solution across multiple households will require an interactive web service solution that is capable of withstanding high web traffic generated from various sources including multiple devices and platforms (i.e. Smart TV Apps, Table App, Web Site App, etc.).
- **Automatic Scalability:** It will prove to be expensive to support the high traffic demands of an interactive web service on a traditional datacenter hosting platform. Consequently, a public cloud-hosted solution that offers a *pay-as-you-go* option might be useful for weathering the proverbial turbulent storms that can be caused by periodic and seasonal spikes in content demands. For example, the needs for movie content

viewing might be higher during the evenings when most people are home relaxing than in the middle of the day when majority of the population is working or in school.

- **High Availability:** Since end users will depend on this engine to suggest the next show that they watch, the solution will need to be able to withstand minor service disruptions and remain fault tolerant.
- **Fast Machine Learning Algorithm:** In determining the similarity score and social influence of a given user's Facebook friends (in support of social intelligence), optimized algorithms will be useful in processing and generating relevant insights quickly.
- **Security and Privacy:** Social media, by default, comes with a looming privacy connotation, and rightfully so. Leveraging a cloud-hosted solution opens up further concerns about privacy and security. End users are going to be concerned about the ubiquitous need for this solution to glean social intelligence data from their Facebook profile. Facebook's OAuth authentication strategy can be used to address issues with authentication but authorization remains a different area of concern.
- **Data Storage:** In this case a relational database management system (RDBMS) will be useful for storing the structured data.

B. Sentiment Analysis in Social Content

While the social profile SMI application described in the previous scenario presents challenges with managing large structured data sets, Social Content Analysis (in the SMI classification spectrum) deals with larger datasets consisting of user-generated content (UGC). The analysis of unstructured social media content data often requires the use of big data storage and analysis techniques. It is estimated that the size of the "digital universe" was 0.18 zettabytes in 2006 – where a zettabyte is equivalent to one billion terabytes [14]. Big Data is here to stay though the growth of unstructured data is at an unprecedented high. In 2012, there was over 30 petabytes of UGC [15] stored, analyzed and processed by Facebook. In the same vein, Twitter was supporting about 175 million tweets daily as of 2012.

It comes with no surprise that some of the architectural concerns for a big data solution that supports SMI might differ slightly from that of a social profile analysis solution that is more focused on analyzing structured data in most cases. Of course, there are hybrid solutions that require the storage and analysis of both types of SMI entities. In evaluating the key characteristics of a social media content solution, we consider the analysis of sentiments in tweets sent out in relation to a recent tweet from a given brand to announce a new product or feature. For example, we consider the kind of reaction (or sentiment) that Microsoft's

unveiling of a new product like XboxOne might garner when it is announced through a tweet on Twitter.com.

Architectural Concerns and Non-Functional Requirements

Some of the unique concerns in this scenario include:

- **Data Storage:** In this case, unstructured data will be processed by a batch process. Data consistency is not as critical in this batch processing scenario as it is in the interactive solution presented in the first scenario
- **Elastic Scalability:** Horizontal scalability is of the essence in this scenario. Making use of a cloud hosted Hadoop solution exposes opportunities to scale up and down seamlessly
- **Fault-Tolerant Storage System:** A graceful fault-tolerant distributed file system like the Hadoop File System (HDFS) proves to be useful when running the solution on commodity hardware [8]
- **Bandwidth:** Network bandwidth is usually a scarce resource in MapReduce implementations [8] though making use of data locality optimization [8] minimizes this concern
- **Efficient Classification Algorithm:** An efficient algorithm for classifying sentiments in the twitter messages is critical.

III. THE REFERENCE ARCHITECTURE

Considering the SMI scenarios described in section III, we designed a technology agnostic reference architecture that takes into account key non-functional requirements and quality attributes.

A. Key Characteristics

Some of the notable characteristics of the reference architecture include the following:

- **Cloud Computing:** By employing a public cloud-hosted solution in our reference architecture, a lot of the heavy-lifting regarding elastic auto-scaling, load-balancing, fault-tolerance and high availability zones across datacenters is intrinsically offloaded to the cloud provider. As a cloud consumer, the SMI application owner might still need to make tradeoff decisions regarding which type of cloud deployment model is best suited for each component in the reference architecture.

For example, for an SMI web service interface, we recommend the use of a Platform-as-a-Service (PaaS) cloud solution in lieu of an Infrastructure-as-a-Service (IaaS) model in order to maximize the promise of auto-scaling and minimize the time investment needed to manage virtual machines and their associated operating system requirements on the cloud platform.

In our studies, we leveraged various services across public clouds including Amazon AWS cloud,

Microsoft's Azure cloud and Google's cloud for various scenarios. Consequently, our proposed architecture transcends any one specific cloud platform.

- **Collective Intelligence Algorithms:** We encourage the use of various machine learning algorithms to extract social media intelligence. Leveraging Apache Mahout for implementing distributed or scalable machine learning algorithms can be useful in solving collaborative filtering, clustering and classification problems. We found it useful to leverage Python as a

programming or scripting language across various cloud implementations.

B. Logical View of the Architecture Model

A logical view of our proposed reference architecture is illustrated in Figure 4. The actors/personas involved in the architecture are also labeled in the illustration. The logical architecture consists of a Presentation or Front-End Tier, a Data Management Tier, and a Storage Tier. These layers and components can be implemented differently for each SMI solution. Though, in most cases all layers are fully utilized.

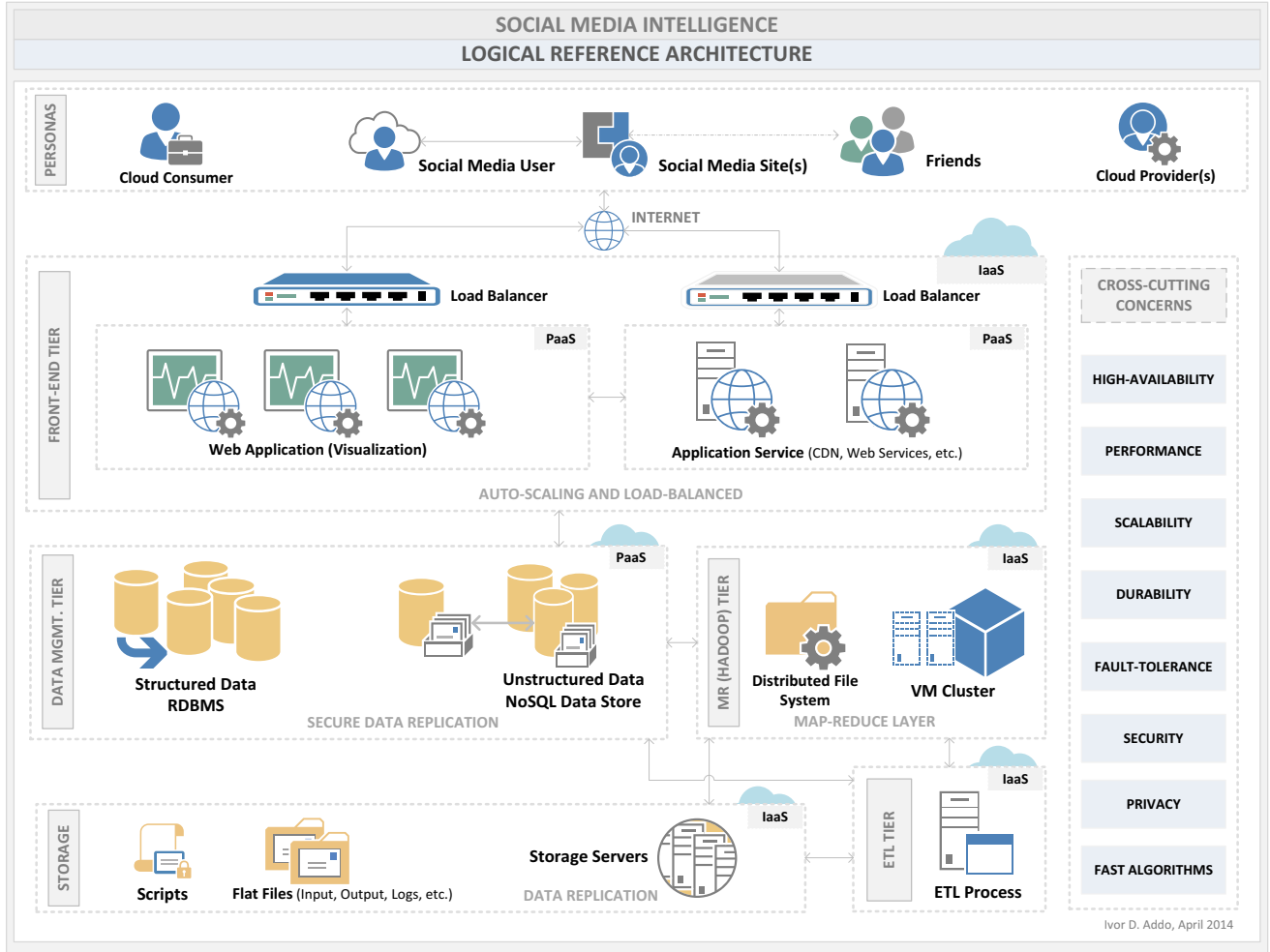


Figure 2. Logical View: Reference Architecture for SMI Solutions

C. Key Cross-Cutting Concerns

Some of the recurring quality attributes that appeared to resonate well with the cross-cutting non-functional requirements identified in our architectural analysis of the case studies presented in section 2 include:

- High Performance
- Elastic Scalability

- High-Availability and Disaster Recovery
- Fault-Tolerance and Durability
- Fast and Scalable Machine Learning Algorithms
- Security and Privacy.

D. Architectural Components and Relationships

Our reference architecture features various architectural components. Notably, we identify some of the most

prominent personas or actors in a cloud-hosted SMI solution as:

- *Cloud Provider*: Represents the cloud vendor who provides cloud services in the Public Cloud. For example, Amazon, Google, Microsoft.
- *Cloud Consumer*: Represents the SMI solution provider who consumes cloud infrastructure and services in a bid to deliver social intelligence solutions.
- *Social Media User*: Representing a consumer of social media who the SMI solution seeks to glean information about.
- *Friends*: In the general sense, these groups of users tend to have a relationship of some sort with the Social Media User. Both parties participate on a Social Media communications channel like an OSN.

We identify a load-balanced and auto-scaling application tier as a layer that represents a set of cloud-hosted infrastructure dedicated to maximizing the performance and availability of the end-user experience with an interactive component of the SMI solution (for example, an SMI web site and/or associated web services that will be responsible for surfacing insights to end-users). We recommend the use of a PaaS solution for this tier to minimize the cost of hardware and system maintenance while benefiting from auto-scaling capabilities. For the data tier, we showcase the implementation of a MapReduce (Hadoop) Layer that can be used to process unstructured data and fed back into either a relational database management system (RDBMS) or a NoSQL solution depending on the needs of the solution, by using an Extract-Transform-Load (ETL) tool. On Amazon AWS cloud, for example, the Amazon EMR (Elastic MapReduce) feature will be used for this component. On the Microsoft Azure cloud, an equivalent solution that implements Apache Hadoop through the Azure HDInsight Service can be employed.

The interactive interface (web services) will communicate directly with the RDBMS or NoSQL data stores to read and write data. Ideally, a PaaS-based NoSQL solution like MongoDB is likely to deliver optimal performance in large data processing for unstructured data. Batch processing (ETL) tools can be employed to load data to the storage layer in the cloud IaaS implementation. The Python scripts can also live on the Storage layer of the cloud solution. Virtual machines (for example, EC2 instances on Amazon AWS) are used to drive the MapReduce processes. These instances can be scaled up and down to meet the requirements of the big data processing task at hand. The following case studies explore how this logical architecture might implement each of the components described in the generic reference model.

IV. CASE STUDY I: A FACEBOOK-INFLUENCED MOVIE RECOMMENDATIONS ENGINE

We employ the collaborative filtering [5] approach by utilizing movie preference data gleaned from Facebook.com, an online social networking (OSN) source for social media. We developed a Microsoft Azure cloud-hosted recommendations engine that is capable of deducing a similarity score between a given user and his Facebook friends. Using the Facebook Social Graph API [12], the preferences of friends with a high similarity score are used to drive the ranking engine. Facebook user preference information pertaining to movie genres and the actual number of “likes” recorded for movies by friends are used to deduce and rank movie suggestions, while taking into account the user’s own preferences and historical viewing behavior. The relation data was stored on Microsoft’s SQL Azure platform.

We simulated a futuristic scenario where the recommendations engine is leveraged at large-scale by multi-family households for recommending movie content by detecting and recognizing multiple users located in front of a smart television (TV) or a large screen tablet to watch a movie. A TV-mounted Microsoft Kinect for Windows sensor is leveraged to aid the simulated smart TV App prototype [13] as shown in Figure 3.

In this scenario, we sort out to employ social media influence intelligence retrieved from user similarity analysis implemented by using the Euclidean Distance Score approach [5] to find commonality between a given user and his Facebook friends. If a friend has a high similarity score then the “likes” or movie preferences of the friend will be ranked higher on the list of recommendations served up by the movie recommendations engine. In addition we explored the application of implementing eigenvector centrality to determine friends who have more influence of a given user. Base on the influence score of a given friend with high social influence, the friends movie preferences would be ranked higher.

We published the web application [16] to the Microsoft Azure PaaS cloud platform using the proposed architectural approach in this literature to test the goodness-of-fit and performance of our template architecture when applied to a sample Social Profile Analysis SMI solution. In this implementation, a web interface using PaaS was used in the Front-End Tier of the logical architecture. The Azure CDN service was employed as a *Content Delivery Network* in the *Application Service* component of our model. SQL Azure was used for RDBMS solution. There were no ETL and MapReduce components involved with this implementation. However, the Azure Storage service was employed in the Storage Tier of our model.

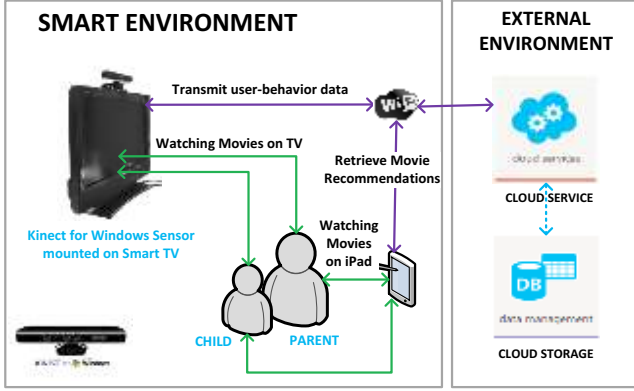


Figure 3. Movie Recommendations Application Scenario [13]

By employing these choices, we were able to inherently achieve our high performance, availability and scalability requirements. However, privacy remains a critical weakness in our prototype implementation.

While we experimented with a number of machine learning algorithms for recommender systems, we found the basic premise of using eigenvalue degree centrality to rank movies quite interesting. For this type of determination of social influence, we consider a naïve degree centrality approach where an *influence function* I_d is represented as follows:

$$I_d(p_i) = \text{degree}(p_i) \quad (1)$$

In equation (1) d represents the degree or number of friends that a user has and the degree function represents the total number of friends connected to a given person, p . Representing the graph as an adjacency matrix, allows us to calculate the Influence of a node as the row-sum of the corresponding row. In Figure 4, $I_d(P1) = 8$; $I_d(P2) = 4$; this represents high degree and low centrality for P1 while P2 is only connected to 4 direct friends but based on their network of friends, P2 might turn out to have the ability to influence more people with his choices and actions on the OSN.

To capture the indirect influence of P2, we explored eigenvector centrality. In this case, a person's social influence is proportional to the total influence of the people to whom he is connected. Assuming the influence function for our eigenvector centrality is represented as I_e , then for friends F1, F2, F3, F4 we can obtain equation (2):

$$I_e(P2) = \frac{1}{\lambda} [I_e(F1) + I_e(F2) + I_e(F3) + I_e(F4)] \quad (2)$$

For a social graph, G with people (vertices) and social relationship (edges), we let A represent an adjacency matrix of G . The N people in the social network can be labeled as $p_1, p_2, \dots, p_n$. We can then generalize the function as equation (3):

$$I_e(p_i) = \frac{1}{\lambda} \sum_{j=1}^N A_{i,j} I_e(p_j) \quad (3)$$

In this case, λ is a function of the social graph with N people (representing vertices) and their relationships (represented as edges) in the *Social Graph*.

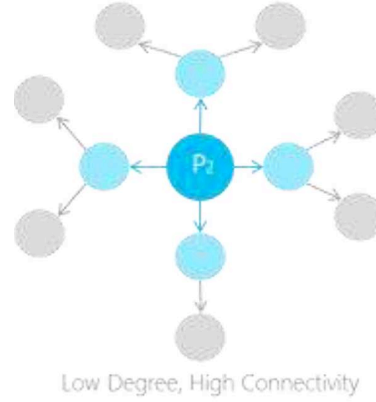
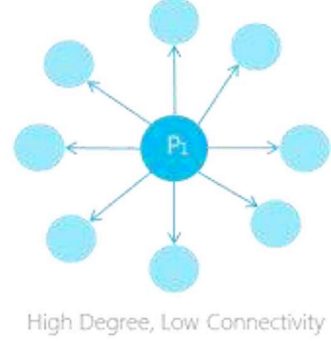


Figure 4. Sample Centrality Diagrams for Social Influence

V. CASE STUDY II: SOCIAL SENTIMENT ANALYSIS ON TWITTER DATA

The Amazon AWS cloud implementation of Hadoop was utilized to execute MapReduce jobs for this big data analysis solution. The Twitter API was utilized to collect tweets regarding a given brand. Using the Naïve Bayes machine learning classification method featured in the Natural Language Toolkit for Python (NLTK) [19], sentiments were classified as either: positive, negative or neutral [20] as illustrated in Figure 5.

In implementing the sentiment analysis prototype, we deployed an instance of the Amazon EMR to represent the MapReduce layer in the reference architecture. There was no user interface for visualization so the front-end tier was not implemented. However, in a future implementation we plan on surfacing visualizations from the EMR batch process output through a web interface. Amazon S3 was used for the storage tier. Input and output files for the MapReduce process along with the python scripts for the mapper tasks were stored on the Amazon S3 service.

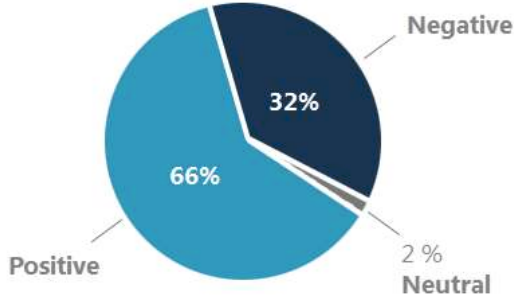


Figure 5. Sample Tweet Sentiment Classification via AWS

In our sentiment analysis, we discovered that real-time sentiment analysis can be a challenging task for highly dynamic textual content that lacks labeled data to support the supervised classifiers. Guerra et al. [31] proposed a model called “opinion holder bias prediction” for real time sentiment analysis. Their approach consists of the following steps: measure the bias of social media users to a specific topic, convert the user bias to textual features for sentiment analysis, and utilize user bias as the basis for building the classifier. We employed these steps in performing sentiment analysis on tweets.

While there are several machine learning algorithms available for this implementation, we implemented the Naïve Bayes classifier for sentiment analysis. The probability model employed in our scenario run assumes that all attribute values v_j of a given document d with K attributes are independent for the category label c , where $P(c)$ and $P(v_j|c)$ represents the class prior and the conditional probability respectively [21]. The “maximum a posteriori” (MAP) classifier is then constructed using the standard function as shown in equation (4) below:

$$c^* = \arg \max_{c \in C} \left\{ P(c) \times \prod_{j=1}^K P(v_j | c) \right\} \quad (4)$$

Our initial test of the classifier was based on real tweet data retrieved from Twitter. Gathering large datasets of tweet data from Twitter turned out to be a daunting task. There are limitations to the amount of data that can be retrieved from Twitter using the Twitter API. As a result, we ended up building a simulation of tweet data using a Python script that employs the Markov Chain algorithm for text generation. This allowed us to produce a large dataset of simulated tweets with sentimental keyword injections that were used to test the classifier on a large-scale AWS Elastic MapReduce (EMR) implementation.

VI. EVALUATION

In evaluating the efficacy of the architectural reference model, we used the following criteria:

- Goodness of Fit analysis of the reference architecture
- Scalability analysis
- Performance analysis.

A. Goodness of Fit Analysis of the Model

We assessed how the design and implementation of the two scenarios described in this literature matches the design of the generalized model. We also compared previously published SMI solution implementations to our model to determine how these solutions could achieve some of the cross-cutting non-functional requirements identified here if they were to be implemented with a cloud-hosted solution that follows our model. In the nine use cases that we studied our generic model was applicable. The model can be employed successfully (with some optional components based on the SMI solution requirements) to achieve high scalability, high performance, and high availability gains.

B. Scalability Analysis

The scalability of the architectural model is of critical importance. In our study, we noted that for horizontal scaling the EMR solution, we could easily adjust the number of nodes based on the workload in question to achieve a linear progression. In our on-premise database instance of SQL Server we were able to achieve vertical scaling by adding more memory and CPU cores to the database server hosting the database instance. However, in scaling out our SQL Azure cloud database we had to implement *sharding* – which allowed us to horizontally partition data across multiple databases.

C. Performance Analysis

While the cloud implementation inherently helps us achieve high performance, we were able to test the performance of SQL Azure queries for the same dataset against that of Google’s BigQuery data store. We noted significant performance improvements when hosting structured data for the movie recommendations engine (378 million records) on BigQuery and running multiple queries simultaneously.

We also performed a speedup analysis (illustrated in figure 6 below) by using the Amazon EMR solution for MapReduce processing in implementing our sentiment analysis solution.

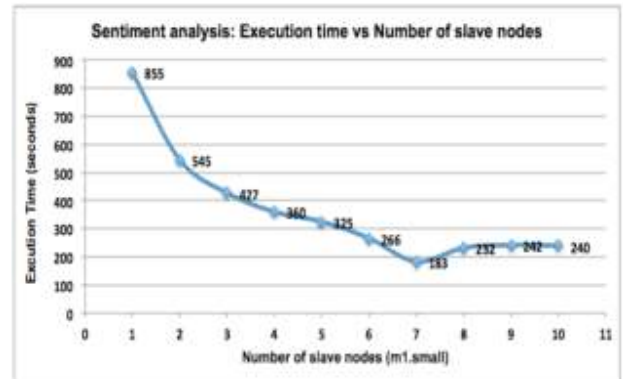


Figure 6. Sentiment Analysis AWS EMR - Speedup Analysis

VII. RELATED WORKS

While several research works have focused on the implementation and analysis of SMI, we are not aware of a reference architecture that can be used for holistically governing some of the major architectural concerns described in this literature. Nonetheless, our work was inspired by recent studies regarding the million Twitter follower fallacy [1] which is focused on social media data analysis, mood and sentiment classification [24] [26] as well as collective intelligence implementations across social media scenarios [3][5].

Our Map-Reduce tasks for Twitter Sentiment Analysis was inspired by several novel research works pertaining to challenges in data sparseness [30] and the complex semantics of tweets. In addition, we found insights in the discussion presented in [31] regarding using limited labelled data for a fully supervised model and the impact of noisy factors (i.e. emoticons, hashtag) in sentiment analysis. Our approach for the Facebook-influenced movie recommendations system was also partially influenced by the study on the different assortativity patterns present in the structure of the Facebook Social Graph [29].

VIII. CONCLUSIONS

While the outburst of social media and big data tools have created numerous opportunities for social intelligence analysis and solutions, there is minimal guidance available today on the best way to build and sustain highly scalable, high availability and high performance SMI solutions. We shared a reference architecture to help guide researchers, solutions architects and developers in the implementation of cloud-based SMI applications. We are hopeful that our reference model will be used in governing future SMI implementations and extended to fit additional needs.

REFERENCES

- [1] M. Cha, H. Haddadi, F. Benevenuto, P. K. Gummadi, "Measuring User Influence in Twitter: The Million Follower Fallacy," *ICWSM*, pp. 10-17, 2010.
- [2] A. Pak, P. Patrick, "Twitter as a Corpus for Sentiment Analysis and Opinion Mining," *LREC*, pp. 1320-1326, 2010.
- [3] M. Russell, "Mining the Social Web," *O'Reilly*, 2012.
- [4] M. R. Morris, J. Teevan, K. Panovich, "A Comparison of Information Seeking Using Search Engines and Social Networks," *ICWSM*, pp. 23-26, 2010.
- [5] T. Segaran, "Programming Collective Intelligence – Building Smart Web 2.0 Applications," *O'Reilly*, 2007.
- [6] I. D. Addo, S. I. Ahamed, W. C. Chu, "Toward Collective Intelligence for Fighting Obesity," *COMPSAC*, 690-695, 2013.
- [7] G. DeCandia, D. Hastorun, M. Jampani, G. Kakulapati, A. Lakshman, A. Pilchin, S. Sivasubramanian, P. Voshall, W. Vogels, "Dynamo: Amazon's Highly Available Key-Value Store," *SOSP*, 2007.
- [8] J. Dean, S. Ghemawat, "MapReduce: Simplified Data Processing on Large Clusters," *OSDI*, 2004.
- [9] H. Liu, D. Orban, "Cloud MapReduce: a MapReduce Implementation on top of a Cloud Operating System," *Cluster, Cloud and Grid Computing*, pp.464-474, May 2011.
- [10] S. Ghemawat, H. Gobioff, S. Leung, "The Google File System," *SOSP*, 2003.
- [11] A. E., Hassan, R. C. Holt, "A Reference Architecture for Web Servers," *Reverse Engineering, 2000. Proceedings. Seventh Working Conference*, pp.150-159, 2000.
- [12] Facebook, "Using Graph API," *Facebook*, n.d.
- [13] I. D. Addo, S. I. Ahamed, S. Yau, A. Buduru, "A Reference Architecture for Improving Security and Privacy in Internet of Things Applications," *Mobile Services*, 2014.
- [14] T. White, "Hadoop: The Definitive Guide," *O'Reilly*, 2012.
- [15] Wikibon, "Big Data Statistics," *Wikibon*, 2012.
- [16] I. Addo, "Movie Recommendations," *HowDoWeFeel*, 2013.
- [17] Amazon, "Amazon EMR," *Amazon AWS*, 2014. Retrieved from <http://aws.amazon.com/elasticmapreduce/>.
- [18] Microsoft, "Features," *Microsoft Azure*, 2014. Retrieved from <http://azure.microsoft.com/en-us/solutions/web/>.
- [19] J. Perkins, "Python text processing with NLTK 2.0 cookbook," *Packt Publishing*, November 2010.
- [20] A.M. Bidgoli, "A Language Independent Text Segmentation Technique Based on Naive Bayes Classifier," *Signal and Image Processing (ICSIP), 2010*, pp. 11-16, 2010.
- [21] F. Peng, X. Huang, D. Schuurmans, S. Wang, "Text Classification in Asian Languages without Word Segmentation," In *IRAL '03*, vol. 11, pp. 41-48, 2003.
- [22] R. D. Zota, A. Fratila, "Toward the Selection of an Enterprise Architecture Model for a Cloud Environment," *Roedumet International Conference (RoEduNet)*, pp. 1-6, 2013.
- [23] B. J. Jansen, M. Zhang, K. Sobel, A. Chowdury, "Micro-blogging as online word of Mouth Branding," *CHI EA*, 2009.
- [24] A. Bermingham, A. F. Smeaton, "Classifying sentiment in microblogs: is brevity an advantage?" In *Proceedings of the 19th ACM international conference on Information and knowledge management, CIKM '10*, pp. 1833-1836, 2010.
- [25] Tang, H. Liu, "Feature Selection with Linked Data in Social Media," In *SDM*, 2012.
- [26] G. Mishne, "Experiments with Mood Classification in Blog Posts," In *1st Workshop on Stylistic Analysis of Text For Information Access*, 2005.
- [27] T. Wilson, J. Wiebe, P. Hoffmann, "Recognizing Contextual Polarity in Phrase-Level Sentiment Analysis," In *Proceedings of HLT/EMNLP 2005*, 2005.
- [28] C. D. Manning, H. Schutze, "Foundations of Statistical Natural Language Processing," *MIT Press*, 1999.
- [29] J. Ugander, B. Karrer, L. Backstrom, C. Marlow, "The Anatomy of the Facebook Social Graph," pp. 4503, 2011.
- [30] X. Hu, N. Sun, C. Zhang, T. S. Chua, "Exploiting internal and external semantics for the clustering of short texts using world knowledge," *CIKM*, pp. 919-928, 2009.
- [31] P. Guerra, A. Veloso, W. Meira Jr., V. Almeida, "From bias to opinion: A transfer-learning approach to real-time sentiment analysis," *SIGKDD*, 2011.
- [32] Adobe, "Social Intelligence Report," *Adobe Digital Index: Q1 - 2014*, 2014.